練習 24：可編輯 ALV——REUSE_ALV_GRID_DISPLAY_LVC

> 授課順序：接在練習 9（Functional ALV）之後。講義見 lec09 第 8 節起（進階篇）。

學習目標

- 會用 `REUSE_ALV_GRID_DISPLAY_LVC` 與 LVC 型別（`lvc_t_fcat` / `lvc_s_layo`），知道跟 slis 版的差異
- 會做**可編輯欄位**與**選取 checkbox**（fieldcat `edit` / `checkbox`）
- 會用 `IS_LAYOUT_LVC-STYLEFNAME` + `lvc_t_styl`：自訂 Fcode 執行後，把已處理列的 checkbox **反灰鎖定**
- 會掛自訂按鈕：PF-STATUS 回呼 + `USER_COMMAND` 回呼 + `selfield-refresh`
- 會讓 `DATA_CHANGED` **事件在「離開已編輯儲存格」當下真的觸發**做即時檢核：`I_GRID_SETTINGS-EDT_CLL_CB = 'X'`（開關）+ `IT_EVENTS` **掛** `SLIS_EV_DATA_CHANGED` **對應的 FORM**（接收），兩者跟 PF-STATUS / USER_COMMAND 是同一套「設定 + FORM 回呼」模式，全程不需要 class 或事件語法

事前準備

1. 建立程式 `ZR_TR24_<你的姓名縮寫>`，套件（Package）`$TMP`；SCARR 要有資料（同 ex09）。
2. **GUI Status（手動）**：SE80 開啟你的程式 → 右鍵 Create → GUI Status，名稱 `STANDARD`——建立時選「**複製**」來源：程式 `SAPLKKBL` 的 Status `STANDARD_FULLSCREEN`（全螢幕 ALV 的標準工具列），複製後在 Application Toolbar 找個空位加 Function Code `ZCONF`、文字「確認」，啟用。
   - 沒建這個 Status 就執行程式會 dump（`SET PF-STATUS` 找不到狀態）——講義 9 的陷阱表第一次實地體驗。

題目需求

情境：航空公司「確認作業」清單——勾選要確認的公司、可填備註，按「確認」後該列狀態變更並**反灰鎖定**，不能再勾、不能再改備註。

1. 資料結構 `ty_row`：`sel`（c1，checkbox）、`carrid`、`carrname`、`remark`（c20，可編輯備註）、`status`（c10）、`celltab TYPE lvc_t_styl`（樣式表欄位）
2. SELECT SCARR 進內表（Internal Table），`status` 先填「未確認」
3. 用 MACRO（ex09 技能）建 **LVC fieldcat**：`SEL`（checkbox + edit）、`CARRID`、`CARRNAME`、`REMARK`（edit）、`STATUS`——注意 `CELLTAB` **不放**進 fieldcat
4. layout（`lvc_s_layo`）：`zebra`、`cwidth_opt`、`stylefname = 'CELLTAB'`
5. `i_grid_settings`（`lvc_s_glay`）：`edt_cll_cb = 'X'`——離開已編輯儲存格觸發 DATA_CHANGED 的開關
6. `it_events`：先宣告事件表的一列 `gs_event TYPE slis_alv_event`，填 `gs_event-name = slis_ev_data_changed`、`gs_event-form = 'DATA_CHANGED'`，再 `APPEND gs_event TO gt_events.`——告訴 wrapper 事件發生時回呼哪個 FORM（開關開了沒掛 FORM 一樣不會執行任何檢核邏輯）
7. 呼叫 `REUSE_ALV_GRID_DISPLAY_LVC`，掛 `i_callback_pf_status_set`、`i_callback_user_command`、`i_grid_settings`、`it_events`

8. `SET_PF_STATUS` 回呼：`SET PF-STATUS 'STANDARD'`（純設定 Status，跟 ex09 一樣單純）
9. `DATA_CHANGED` 回呼——**FORM 介面固定**：`FORM data_changed USING pr_data_changed TYPE REF TO cl_alv_changed_data_protocol.`。檢查 `pr_data_changed->mt_good_cells`：**只有 `REMARK` 這欄的新值含 `!`** 才呼叫 `pr_data_changed->add_protocol_entry( ... )` 報錯；其他情況什麼都不做（讓修改正常生效）
10. `USER_COMMAND` 回呼：處理 `ZCONF`——把 `sel = 'X'` 的列：`status` 改「已確認」、往 `celltab` 塞 `SEL` 與 `REMARK` 兩筆 `mc_style_disabled`、最後 `selfield-refresh = 'X'`
11. 完整驗證流程（**照順序做，先驗證「正常輸入不受影響」再驗證「異常輸入被擋」**）：
    1. 在任一列備註打普通文字（如 `SDFDF`，不含 `!`）→ 點別列 → **不會有任何提示**，備註正常保留，可以繼續操作
    2. 勾兩列 + 填備註（不含 `!`）→ 按「確認」→ 該兩列狀態變更、checkbox 與備註**反灰**
    3. 再試著點反灰的 checkbox（點不動）
    4. 在**未反灰**的列備註打 `abc!`（含驚嘆號）→ 點別格 → 錯誤紀錄立刻彈出——只有這種情況才會擋

預期結果

- ALV 顯示五欄，SEL 是可勾的 checkbox、REMARK 可編輯，工具列有標準功能 + 自訂「確認」按鈕
- **備註打普通文字、離開儲存格：完全無感，正常繼續操作**——`DATA_CHANGED` 每次離開已編輯儲存格都會觸發，但檢核邏輯只挑 `REMARK` 含 `!` 的情況報錯，其餘一律放行，**不是「只要輸入備註就會被擋」**
- 按「確認」後：勾選列的狀態=已確認，該列 SEL/REMARK 反灰不可再編輯；未勾選列不受影響
- 備註輸入含 `!` 的值、**離開儲存格當下**即彈出錯誤紀錄（把 `it_events` 那行註解掉重跑對照：離開儲存格就完全不會觸發任何檢核，含 `!` 也不會報錯）

思考題

1. `DATA_CHANGED` 每次離開已編輯儲存格都會觸發（不管內容合不合法），但畫面上使用者只在「含 `!`」時才看到錯誤——這個「觸發」與「報錯」分成兩層的設計，比起「只要編輯就一律跳提示」好在哪裡？
2. 如果需求是「REMARK 不可空白」，`data_changed` 這個 FORM 要怎麼改條件？
3. `EDT_CLL_CB` 與 `IT_EVENTS` 兩者只設一個會發生什麼？（提示：分別對照「有開關沒人接」與「有人接但開關沒開」）
4. 反灰用的 `celltab` 是「每一列一張樣式表」——如果需求改成「整欄永遠唯讀」，用 fieldcat 的什麼欄位做更簡單？（提示：不給 `edit` 就好——style 是給「逐列/逐格動態控制」用的）

答案

見 `zr_tr24_alv_lvc.prog.abap`（SAP 端程式 `ZR_TR24_ALV_LVC`；GUI Status `STANDARD` 需依事前準備手動建立，無快照）。